Part 3 – Complete System Workflow & Communication

11. End-to-End System Workflow

11.1 Introduction

The Campus Threat Intelligence Platform consists of multiple independent modules, each responsible for solving a specific part of the phishing detection problem. While every module performs its own task, the true strength of the platform lies in the seamless communication between them.

Instead of functioning as isolated components, every module contributes to a continuous security pipeline that begins with the discovery of a suspicious website and ends with protecting the student before credentials are stolen.

This chapter explains the complete lifecycle of a phishing detection event, illustrating how information flows through the platform from start to finish.

11.2 Overall Workflow

The complete execution pipeline of the Campus Threat Intelligence Platform is illustrated below.

```
                 INTERNET

                      │

                      ▼

          Threat Discovery Engine

                      │

                      ▼

      Candidate Website Collected

                      │

                      ▼

       Metadata + Screenshot + DOM

                      │

                      ▼

          AI Similarity Engine

                      │

                      ▼

           Digital Twin Engine

                      │

                      ▼

         Similarity Comparison

                      │

                      ▼

          Risk Scoring Engine

                      │

          ┌───────────────────────┴───────────────────────┐
```

```
         |           |

         ▼           ▼
```

Threat Database Browser Extension

```
         |           |

         ▼           ▼
```

Administrator Dashboard Student Warning

```
Every phishing website passes through this pipeline before a final decision
is generated.

---

# 11.3 Stage 1 – Threat Discovery

The workflow begins with the Threat Discovery Engine.

At regular intervals, the crawler searches predefined internet sources for
webpages that appear to be associated with institutional login portals.

For the hackathon MVP, the crawler focuses on websites containing keywords
such as:
```

YCCE ERP
YCCE Login
YCCE Student Portal
YCCE Webmail
YCCE Scholarship

```
Whenever a potentially relevant webpage is discovered, the crawler downloads
the webpage and prepares it for analysis.

At this stage, no security decision is made.

The webpage is simply collected.

---

# 11.4 Stage 2 – Website Rendering

Modern websites rely heavily on JavaScript.

Simply downloading HTML is insufficient because many webpage components
appear only after JavaScript execution.

Therefore the crawler launches a real browser instance.

The browser loads the webpage exactly as a student would experience it.
```

```
Once rendering is complete, the crawler captures:

· HTML

· Rendered DOM

· CSS

· JavaScript

· Images

· Screenshot

· SSL Metadata

· Network Requests

This information forms the candidate webpage package.

---

# 11.5 Stage 3 – Candidate Website Creation

After rendering, the crawler creates a Candidate Website object.

The object contains every piece of information required for later analysis.

Example:
```

Candidate Website
Website ID
URL
Screenshot
DOM
HTML
CSS
JavaScript
SSL
Metadata
Timestamp

```
The Candidate Website is then inserted into the Threat Queue.

---

# 11.6 Stage 4 – Digital Twin Retrieval

The AI Similarity Engine cannot begin analysis until it knows what the
official website looks like.

It therefore requests the corresponding Digital Twin.

Example:
```

Candidate Website
↓
ERP Login
↓
Request Official Digital Twin
↓
Digital Twin Engine
↓
Return Official Fingerprint

```
The official Digital Twin now becomes the reference model.

---

# 11.7 Stage 5 – Multi-Layer Similarity Analysis

The AI Similarity Engine performs several independent analyses.

Instead of making an immediate decision, each analyzer evaluates one
characteristic of the webpage.

These include:

Visual Analyzer

DOM Analyzer

CSS Analyzer

Form Analyzer

Logo Analyzer

SSL Analyzer

JavaScript Analyzer

URL Analyzer

Each analyzer generates an independent similarity score.

Example:
```

Visual
98%

DOM
96%
Form
99%
Logo
100%
JavaScript
94%
SSL
87%
URL
18%

> These scores represent evidence rather than final decisions.
>
> ---
>
> # 11.8 Stage 6 – Evidence Fusion
>
> Once every analyzer completes its task, the AI Similarity Engine combines all evidence into a single feature vector.
>
> Instead of evaluating screenshots, DOM, and SSL independently, the system constructs a unified representation of the webpage.
>
> This representation becomes the input for the Risk Scoring Engine.
>
> ---
>
> # 11.9 Stage 7 – Risk Calculation
>
> The Risk Scoring Engine receives the complete feature vector.
>
> Each feature contributes to the final phishing probability.
>
> Example:

Visual
98%
Weight
25%
Contribution
24.5
DOM
96%
Weight
20%
Contribution
19.2
...
Overall Risk
95.8%

> The engine also calculates confidence.
>
> Example:

Risk
95%
Confidence
98%

```
---

# 11.10 Stage 8 – Explainable AI Report

The platform now converts numerical scores into human-readable intelligence.

Example:
```

Risk Score
96%
Reasons
Copied Institutional Logo
Highly Similar DOM
Credential Submission Redirected
Suspicious Domain
Recent SSL Certificate
Recommendation
Do Not Enter Credentials

```
This report is understandable even for non-technical users.

---

# 11.11 Stage 9 – Database Storage

Every completed analysis is stored permanently.

The stored information includes:

Website

Risk Score

Similarity Scores

Evidence

Digital Twin Version

Screenshot

Timestamp

Threat Status

Administrator Notes

Historical storage allows future investigations and trend analysis.

---

# 11.12 Stage 10 – Browser Protection

Whenever a student opens the suspicious webpage, the Browser Extension
immediately contacts the backend.
```

Student
↓
Open Website
↓
Browser Extension
↓
Backend
↓
Already Analysed?
↓
Yes
↓
Return Risk
↓
Warning

```
If the webpage has already been analysed by the platform, the response is
almost instantaneous.

---

# 11.13 Stage 11 – Credential Protection

When the student presses the Login button, the Browser Extension intercepts
the submission event.

If the webpage has been classified as High Risk or Critical, credential
submission is temporarily blocked.

Instead of silently preventing login, the extension explains why the website
is dangerous.

The student can then leave the webpage without exposing institutional
credentials.

---

# 11.14 Stage 12 – Administrator Notification

Whenever a High Risk website is detected, the platform automatically
generates an incident.

The Administrator Dashboard receives:

Website

Risk

Screenshot

Evidence

Timeline

Similarity Report

Administrators therefore become aware of phishing attacks immediately rather
than waiting for student complaints.

---

# 11.15 Internal Module Communication

The following diagram illustrates how every module communicates.
```

```
Threat Discovery Engine

↓

Threat Queue

↓

AI Similarity Engine

↓

Digital Twin Engine

↓

Risk Engine

↓

Threat Database

↓

Dashboard API

↓

Administrator Dashboard

↓

Browser Extension API

↓

Student Protection
```

Every module performs exactly one responsibility.
This separation greatly improves maintainability.

11.16 Why This Architecture?

Many phishing detection systems attempt to perform every task inside the browser extension.
This creates several problems.
The extension becomes large.
AI processing becomes slow.
Updating detection algorithms requires extension updates.
Resource usage increases.
The Campus Threat Intelligence Platform follows a distributed architecture instead.
Heavy computation remains on the backend.
The extension remains lightweight.
AI models can evolve independently.
Administrators receive centralized visibility.
Students receive real-time protection.
This architecture is significantly more scalable and suitable for enterprise deployment.

11.17 Complete Lifecycle Example

The following example demonstrates the complete execution pipeline.

```
Attacker Creates

↓

ycce-erp-login.xyz

↓

Threat Discovery Engine Finds Website

↓

Website Rendered

↓

Screenshot Captured

↓

Digital Twin Loaded

↓

AI Similarity Analysis

↓

Risk

96%

↓

Threat Stored

↓

Dashboard Alert Generated

↓

Student Opens Website

↓

Extension Queries Backend

↓

Risk Returned

↓

Credential Submission Blocked

↓

Student Protected
```

This represents one complete phishing detection cycle within the Campus Threat Intelligence Platform.

11.18 Chapter Summary

The Campus Threat Intelligence Platform operates as a continuous cybersecurity pipeline rather than a collection of isolated tools.
Every module contributes a specific capability, beginning with website discovery and ending with credential protection.
By separating responsibilities across specialized modules, the platform achieves scalability, maintainability, explainability, and real-time phishing defense.
The next chapter will describe the internal database architecture, API design, and communication mechanisms that allow these modules to operate as a unified distributed system.

# New Module – Evidence Extraction Engine

# 6.5 Evidence Extraction Engine

## 6.5.1 Introduction

The Evidence Extraction Engine acts as the preprocessing layer of the Campus Threat Intelligence Platform. It serves as the bridge between the Threat Discovery Engine and the AI Similarity Engine.

The Threat Discovery Engine is responsible for collecting raw information from suspicious websites. This information includes HTML source code, rendered DOM, CSS files, JavaScript files, screenshots, SSL certificates, metadata, and network requests.

However, raw webpage data cannot be directly consumed by Artificial Intelligence models.

Raw HTML may contain unnecessary whitespace.

CSS files may contain thousands of unused classes.

JavaScript files may include third-party libraries that are irrelevant to phishing detection.

Screenshots may contain browser toolbars or unnecessary margins.

If this raw data were sent directly to the AI Similarity Engine, the system would become inefficient, computationally expensive, and significantly less accurate.

To solve this problem, the Campus Threat Intelligence Platform introduces a dedicated preprocessing layer known as the Evidence Extraction Engine.

The objective of this module is to transform raw webpage information into structured evidence that can be efficiently analyzed by Artificial Intelligence.

Rather than making security decisions, the module prepares clean, normalized, and meaningful features for later analysis.

# 6.5.2 Why Do We Need This Module?

Suppose the crawler downloads the following webpage.

```
HTML

18,000 lines

CSS

7 Files

JavaScript

12 Files

Images

35

Fonts

6

DOM Nodes

3,400
```

Sending all this information directly to the AI model would be inefficient.

Instead,

the Evidence Extraction Engine processes everything first.

```
Raw Website

↓

Clean HTML

↓

Normalize DOM

↓

Extract Login Form

↓

Detect Logo

↓

Crop Screenshot

↓

Extract CSS Features

↓

Extract JavaScript Behaviour

↓
```

Generate Structured Evidence

↓

AI Similarity Engine
\`\`\`

This dramatically reduces computational complexity while improving detection accuracy.

# 6.5.3 Responsibilities

The Evidence Extraction Engine is responsible for transforming raw webpage resources into machine-readable security features.

Its primary responsibilities include:

- Cleaning HTML

- DOM Normalization

- Screenshot Processing

- Logo Detection

- Form Extraction

- CSS Feature Extraction

- JavaScript Behaviour Extraction

- Metadata Normalization

- Feature Vector Generation

Unlike the AI Similarity Engine, this module performs no classification.

Its responsibility is only feature preparation.

# 6.5.4 HTML Normalization

Downloaded HTML often contains unnecessary information.

Examples include:

Comments

Tracking Scripts

Analytics

Advertisements

Whitespace

Random IDs

Dynamic Tokens

These elements do not contribute to phishing detection.

The HTML Normalizer removes unnecessary components while preserving structural information.

The result is a simplified representation of the webpage.

# 6.5.5 DOM Normalization

Every browser generates slightly different DOM structures.

Some webpages also create temporary DOM elements during rendering.

The DOM Normalizer removes unstable elements and produces a consistent structure.

Examples include:

Dynamic IDs

Random Class Names

Session Tokens

Advertisement Containers

Tracking Elements

Hidden Analytics Components

The normalized DOM allows fair comparison against the Digital Twin.

# 6.5.6 Screenshot Processing

The crawler captures a screenshot of the webpage.

However,

raw screenshots often contain unnecessary browser elements.

The Screenshot Processor performs several operations.

Browser UI Removal

Margin Cropping

Resolution Standardization

Image Compression

Noise Reduction

Color Normalization

The resulting screenshot becomes the input for Visual Similarity Analysis.

# 6.5.7 Logo Extraction

Rather than comparing the entire webpage,

the platform identifies the institutional logo separately.

The Logo Extractor locates the largest branding element near the top of the webpage.

After detection,

the logo is cropped,

resized,

and converted into an embedding.

Later,

the AI Similarity Engine compares only logo embeddings instead of full screenshots.

This significantly improves accuracy.

# 6.5.8 Login Form Extraction

The platform focuses specifically on authentication pages.

Therefore,

instead of analyzing the entire webpage equally,

the Evidence Extraction Engine identifies the primary login form.

The following information is extracted:

Username Field

Password Field

Input Order

Placeholder Text

Labels

Buttons

Checkboxes

Forgot Password Links

Submission Endpoint

This structured representation becomes the Form Fingerprint.

# 6.5.9 CSS Feature Extraction

Instead of sending complete CSS files,

the module extracts visual characteristics.

Examples include:

Primary Color

Secondary Color

Font Family

Border Radius

Shadow Style

Button Colors

Background Color

Layout Style

Typography

Spacing

Only meaningful visual information is retained.

# 6.5.10 JavaScript Behaviour Extraction

JavaScript often contains thousands of lines of code.

Most of it is unrelated to phishing detection.

The JavaScript Extractor focuses only on authentication behaviour.

Examples include:

Form Submission

Credential Collection

AJAX Requests

Network Requests

Cookie Access

Local Storage

Clipboard Access

Redirect Logic

This greatly reduces processing requirements.

# 6.5.11 Metadata Normalization

Metadata collected by the crawler originates from multiple sources.

The module converts all metadata into a standardized format.

Examples include:

SSL Information

Domain Age

Registrar

Hosting Provider

IP Address

HTTP Headers

Security Headers

Response Codes

Redirect Chains

Standardized metadata simplifies later analysis.

# 6.5.12 Feature Vector Generation

After preprocessing completes,

all extracted information is converted into a structured feature vector.

Instead of sending multiple independent files,

the AI Similarity Engine receives a unified object.

Example:

```
Website Features

{

Visual Features

DOM Features

CSS Features

Logo Features

Form Features

JavaScript Features

SSL Features

Metadata Features

}
```

This object represents the digital identity of the candidate webpage.

# 6.5.13 Communication Flow

```
Threat Discovery Engine

↓

Raw Website

↓

Evidence Extraction Engine
```

↓

Normalized Features

↓

AI Similarity Engine

↓

Risk Engine
```

The AI never receives raw webpage resources.

It only receives structured evidence generated by this module.

---

# 6.5.14 Advantages

Introducing the Evidence Extraction Engine provides several benefits.

It reduces AI computation.

It standardizes webpage data.

It improves similarity accuracy.

It simplifies future AI model upgrades.

It separates preprocessing from intelligence.

It enables independent optimization of feature extraction without modifying the AI engine.

---

# 6.5.15 Module Summary

The Evidence Extraction Engine serves as the preprocessing backbone of the Campus Threat Intelligence Platform.

It transforms raw webpages into structured, normalized, and meaningful evidence before AI analysis begins.

By separating feature engineering from Artificial Intelligence, the platform becomes cleaner, more scalable, easier to maintain, and significantly closer to enterprise-grade cybersecurity architectures.

# Chapter 12 – Proposed Technology Stack

---

# 12.1 Introduction

Selecting the right technology stack is one of the most important architectural decisions in the development of the Campus Threat Intelligence Platform (CTIP). Every technology used in this project has been selected based on the specific requirements of the system rather than popularity or personal preference.

The platform combines multiple domains including browser extensions, web crawling, artificial intelligence, computer vision, backend APIs, real-time dashboards, and database systems. Since every component has different responsibilities, no single framework or programming language can efficiently solve every problem.

The objective of this chapter is to explain why each technology has been selected, what role it performs within the system, and why it is more suitable than alternative technologies.

---

# 12.2 Overall Technology Stack

| Layer | Technology |
|--------|------------|
| Frontend Dashboard | React + TypeScript + Tailwind CSS |
| Browser Extension | Chrome Extension (Manifest V3) |
| Backend API | FastAPI |
| Programming Language | Python |
| Web Crawler | Playwright |
| Computer Vision | OpenCV |
| Image Embedding Model | CLIP |
| Database | PostgreSQL |
| Cache | Redis |
| Authentication | JWT |
| Real-Time Communication | WebSockets |
| Deployment | Docker |
| Frontend Hosting | Vercel |
| Backend Hosting | Render |
| Database Hosting | Supabase PostgreSQL |
| Redis Hosting | Upstash |

---

# 12.3 Frontend

## React

The Administrator Dashboard is a highly interactive interface that continuously displays cybersecurity intelligence collected by different modules of the platform. Administrators need to monitor phishing campaigns, inspect suspicious websites, manage Digital Twins, review incident reports, and observe live threat activity.

React has been selected because it follows a component-based architecture that makes the interface modular, reusable, and maintainable.

Each feature of the dashboard can be developed as an independent component. For example, the Threat Timeline, Active Threat Table, Incident Viewer, Digital Twin Manager, Analytics Dashboard, and Notification Panel can all be built as reusable components.

This modular approach improves development speed while simplifying future maintenance.

### Why React instead of Angular?

Angular is a complete framework designed primarily for very large enterprise applications. While powerful, it introduces additional complexity through modules, dependency injection, decorators, and configuration.

For a one-week hackathon, Angular increases development time without providing significant advantages for this project.

React provides a much simpler development experience while remaining highly scalable.

### Why React instead of Vue?

Vue is lightweight and beginner-friendly, but React has a significantly larger ecosystem, stronger community support, and wider adoption in enterprise cybersecurity products.

Many commercial dashboards used in security products are built with React due to its flexibility and mature ecosystem.

---

## TypeScript

The Campus Threat Intelligence Platform contains multiple interacting modules that exchange structured data through APIs.

Using plain JavaScript increases the possibility of runtime errors due to incorrect object structures or missing properties.

TypeScript provides compile-time type checking, improving reliability and maintainability.

Benefits include:

- Better IntelliSense
- Early bug detection
- Strong API contracts
- Easier refactoring
- Improved code readability

For a security-focused application, reducing runtime errors is particularly important.

---

## Tailwind CSS

The Administrator Dashboard contains numerous cards, tables, forms, statistics panels, graphs, and navigation components.

Tailwind CSS enables rapid UI development using utility classes instead of writing large custom CSS files.

Advantages include:

- Faster development
- Consistent design system
- Responsive layouts
- Easy customization
- Reduced CSS maintenance

This significantly accelerates frontend development during the hackathon timeline.

---

# 12.4 Browser Extension

## Chrome Extension (Manifest V3)

The Browser Extension is responsible for protecting students while they browse institutional login pages.

Manifest V3 is the latest extension standard supported by Chromium-based browsers and introduces several security improvements over previous versions.

Advantages include:

- Improved security model
- Background Service Workers
- Reduced memory usage

- Better permission management
- Long-term browser compatibility

The extension architecture also makes future support for Microsoft Edge, Brave, and Opera straightforward because they are Chromium-based browsers.

---

# 12.5 Backend

## FastAPI

The backend acts as the central communication layer of the Campus Threat Intelligence Platform.

Its responsibilities include:

- Receiving requests from the Browser Extension
- Managing Digital Twins
- Running AI analysis
- Calculating phishing risk
- Providing APIs for the Administrator Dashboard
- Managing authentication
- Storing cybersecurity intelligence

FastAPI was selected because it is specifically designed for building high-performance asynchronous APIs.

It integrates naturally with Python's AI ecosystem and automatically generates OpenAPI documentation, simplifying both development and testing.

### Why FastAPI instead of Express.js?

Express.js is an excellent backend framework for JavaScript applications.

However, the majority of Artificial Intelligence libraries used in this project are written for Python.

Choosing Express would require a separate Python microservice for AI processing, increasing architectural complexity.

FastAPI allows both the backend APIs and AI modules to exist within the same ecosystem.

This reduces complexity and simplifies deployment.

### Why FastAPI instead of Django?

Django is a full-stack framework that includes many features unnecessary for this project, such as template rendering and an administrative interface.

The Campus Threat Intelligence Platform is API-first rather than template-based.

FastAPI offers:

- Better asynchronous performance
- Lower overhead
- Cleaner REST API development
- Easier AI integration
- Automatic API documentation

These characteristics make FastAPI more suitable.

---

# 12.6 Programming Language

## Python

Python serves as the primary programming language for the backend, AI modules, Digital Twin generation, Evidence Extraction, and Risk Analysis.

Python has the strongest ecosystem for Artificial Intelligence, Computer Vision, Machine Learning, and cybersecurity automation.

Advantages include:

- Rich AI ecosystem
- Mature Computer Vision libraries
- Excellent web automation support
- Large cybersecurity community
- Rapid development speed

Because almost every intelligence component of the platform depends on AI libraries, Python is the natural choice.

---

# 12.7 Threat Discovery Engine

## Playwright

The Threat Discovery Engine must render webpages exactly as they appear to students.

Modern phishing websites frequently rely on JavaScript to dynamically generate content.

Traditional HTML download methods cannot execute JavaScript.

Playwright launches a real browser instance capable of executing JavaScript, rendering dynamic pages, capturing screenshots, extracting rendered DOM structures, and recording network activity.

This makes Playwright ideal for phishing webpage analysis.

### Why Playwright instead of Selenium?

Although Selenium is widely used for browser automation, it was primarily designed for automated testing.

Playwright offers:

- Faster execution
- Better JavaScript rendering
- Improved screenshot capabilities
- Modern browser support
- Simpler programming model

These advantages make Playwright more appropriate for threat discovery.

---

# 12.8 Computer Vision

## OpenCV

OpenCV is responsible for preprocessing webpage screenshots before they are analyzed by AI.

Tasks include:

- Image normalization
- Screenshot cropping
- Logo extraction
- Resolution standardization
- Noise reduction

OpenCV is lightweight, highly optimized, and integrates seamlessly with Python.

---

# 12.9 Visual Similarity Analysis

## CLIP

Comparing webpage screenshots pixel-by-pixel is unreliable because attackers often make small visual modifications.

Instead, the platform uses CLIP to convert screenshots into semantic embeddings.

These embeddings capture the meaning and overall appearance of the webpage rather than exact pixels.

Advantages include:

- Robust similarity detection
- Better generalization
- No custom training required
- High-quality visual understanding

### Why CLIP instead of a Custom CNN?

Training a Convolutional Neural Network requires thousands of labeled phishing webpage images.

Such a dataset is not available within the hackathon timeline.

CLIP is a pre-trained multimodal model capable of producing high-quality visual embeddings without additional training.

This dramatically reduces development effort.

---

# 12.10 Database

## PostgreSQL

The Campus Threat Intelligence Platform stores highly structured information including:

- Threats
- Digital Twins
- Similarity Reports
- Administrators
- Notifications
- Incidents
- Historical Analysis

These entities have many relationships.

PostgreSQL provides:

- ACID transactions
- Excellent relational modeling
- Powerful indexing

- JSON support
- High reliability

These capabilities make it an excellent choice.

### Why PostgreSQL instead of MongoDB?

MongoDB performs well for flexible document storage.

However, this project contains many interconnected entities that naturally fit a relational database.

Examples include:

- One Threat has many Evidence records.
- One Digital Twin may relate to many Threat Analyses.
- One Administrator receives many Notifications.

Managing these relationships is considerably simpler using PostgreSQL.

---

# 12.11 Redis

Repeatedly analyzing the same phishing website wastes computational resources.

Redis stores frequently requested analysis results in memory.

Benefits include:

- Extremely fast lookup
- Reduced AI computation
- Lower API response time
- Improved scalability

This significantly improves Browser Extension performance.

---

# 12.12 Authentication

## JWT

The Administrator Dashboard requires secure authentication without maintaining server-side sessions.

JSON Web Tokens provide:

- Stateless authentication
- Secure authorization
- Easy integration with REST APIs
- Excellent scalability

JWT is widely adopted across modern enterprise applications.

---

# 12.13 Real-Time Communication

## WebSockets

Threat intelligence changes continuously.

Administrators should not manually refresh the dashboard.

WebSockets enable the backend to immediately push newly detected threats to connected administrators.

Applications include:

- Live threat feed
- Instant notifications
- Real-time incident updates
- Dynamic dashboard synchronization

---

# 12.14 Containerization

## Docker

The Campus Threat Intelligence Platform consists of multiple independent services.

Docker packages every service with its dependencies, ensuring consistent execution across development, testing, and production environments.

Advantages include:

- Environment consistency
- Simplified deployment
- Easy collaboration
- Dependency isolation
- Improved scalability

---

# 12.15 Deployment Strategy

| Component | Platform |
|-----------|----------|
| Administrator Dashboard | Vercel |
| Backend API | Render |
| PostgreSQL Database | Supabase |
| Redis Cache | Upstash |
| Browser Extension | Chrome Web Store / Load Unpacked |

This deployment strategy minimizes infrastructure complexity while remaining sufficiently scalable for demonstration and future production deployment.

---

# 12.16 Chapter Summary

The selected technology stack balances rapid hackathon development with production-grade architecture.

React provides a modular dashboard, FastAPI and Python power the backend intelligence, Playwright enables realistic webpage rendering, OpenCV and CLIP perform visual analysis, PostgreSQL manages structured cybersecurity intelligence, Redis improves performance, Docker simplifies deployment, and WebSockets deliver real-time communication.

Each technology has been selected based on technical suitability rather than popularity, ensuring that every component of the Campus Threat Intelligence Platform fulfills its responsibilities efficiently while remaining scalable for future enhancements.
```